LEXEASE — PROJECT CONTEXT SNAPSHOT

For Claude Code: drop this file in your project root and reference it at the start of every session.

Generated from: full project planning conversation (Claude.ai)

Last updated: July 2026

---

1. IDENTITY

Field	Value
Official Title	"Developing a System for Legal Document Simplification and Clause Risk Analysis Using AI"
Internal Name	LexEase
Type	CSE Final Year Project — Group of 5
Nature	Software Only — No hardware, no paid APIs
Year	2025–26
Degree	B.E. / B.Tech Computer Science & Engineering

---

2. PROJECT DESCRIPTION (one para)

LexEase is a web application that lets ordinary Indian citizens upload legal documents (PDFs — rental agreements, court notices, employment contracts, government forms) and receive: (1) a plain-language summary of the whole document, (2) clause-by-clause risk classification (Low / Medium / High) with explanations, (3) an interactive Q&A session where users ask questions grounded in their uploaded document, and (4) optional output in Hindi or Marathi. No document is persistently stored. Built entirely on free and open-source tools.

---

3. TEAM STRUCTURE

Member	Role
Member 1	Frontend — Upload UI, clause display, risk visualization
Member 2	Frontend — Interactive Q&A chat + multilingual toggle
Member 3	Backend — FastAPI, routing, session management, auth
Member 4	AI/LLM — PDF parsing pipeline, Gemini prompt engineering, LangChain
Member 5	Data/QA/Docs — Testing, dataset curation, literature survey, project report

---

4. CONFIRMED TECH STACK

All tools are free and open-source unless noted.

Layer	Tool	Why This / Not Others
Frontend	React.js + Tailwind CSS	Component model suits clause-by-clause UI; parallel dev by 2 members; best resume value
Backend	Python 3.11 + FastAPI	Async-native (waits on Gemini API calls); auto Swagger docs; same language as AI layer
PDF Parsing	PyMuPDF (fitz) + pdfplumber (fallback)	Handles multi-column Indian legal PDFs; both pip install, no API key
AI Engine	Google Gemini 2.0 Flash (free tier)	1,500 req/day, no credit card, 1M token context window
LLM Orchestration	LangChain	Prompt templating + conversational memory for Q&A without building from scratch
Database	PostgreSQL	Free, open-source; stores session metadata only (NOT document text)
Translation	Google Translate API (free tier)	Supports Hindi + Marathi; 500K chars/month free

Layer	Tool	Why This / Not Others
Deployment	Render (free tier)	Genuine free tier (750 hrs/month); no credit card needed
Version Control	Git + GitHub	Team collaboration; GitHub Actions for CI/CD
Containerization	Docker	Consistent dev environment across 5 machines

Rejected alternatives (important for justification questions):

- Railway: dropped free tier 2023 — needs credit card. Do NOT use.
- OpenAI/Anthropic: pay-per-token, no meaningful free tier.
- Local LLMs (Ollama/Llama): needs GPU — violates no-expensive-hardware rule.
- Django over FastAPI: unnecessary bloat for pure API + AI pipeline.
- MongoDB over PostgreSQL: data is relational (sessions, categories); Mongo adds no benefit.
- DeepL: doesn't support Hindi or Marathi — hard disqualifier.
- Vercel: frontend only, can't run FastAPI backend.

---

5. CRITICAL CONSTRAINT — GEMINI FREE TIER PRIVACY

⚠ **This MUST be disclosed honestly — don't oversell privacy.**

- Gemini FREE tier: Google's terms allow submitted content to be used for model improvement + human review.
- Gemini PAID tier / Vertex AI: contractual no-training-on-data guarantee.
- LexEase uses the FREE tier → project demo must use SAMPLE/SYNTHETIC documents, not real personal legal documents.
- Upload screen must carry a clear disclaimer: "Academic prototype — do not upload sensitive real documents."
- Mitigation: PII masking step (regex to redact phone numbers, ID patterns, names) before text is sent to the API.
- If asked by committee: frame as a deliberate, scoped trade-off for an academic build. Paid tier = clear next step for a real product.

---

6. PROFESSOR'S TITLE (exact wording)

> "Developing a System for Legal Document Simplification and Clause Risk Analysis Using AI"

---

7. PROFESSOR'S INSTRUCTIONS (verbatim requirements)

A. Literature Survey Table

Prepare a Word (.docx) file with a table having these columns:

1. Sr. No.
2. Title of existing paper/application
3. Problem solved / work done
4. Methodology used
5. Technique / Algorithms used
6. Result evaluation parameters
7. Result value (%) for each parameter
8. Features
9. Limitations

Quota: minimum 30 research papers + 10 applications/websites

B. Project Planning (after survey)

Document the following:

- Project modules
- Software and hardware requirements
- Overall methodology (how it will work end-to-end)
- Coding languages
- Database needed
- Dataset required
- Result evaluation parameters
- Expected results (with % targets)
- Applications of the project

C. Feature Enhancement Rule

The final system must implement: **existing features from surveyed work + additional**

novel features + novel techniques/algorithms to achieve better result/efficiency/performance than existing systems.

---

8. PLANNED FEATURES

Core (must have):

- PDF upload + text extraction (PyMuPDF)
- Whole-document plain-language summary
- Clause segmentation + per-clause risk classification (Low/Medium/High)
- Plain-language explanation per clause
- Interactive document-grounded Q&A (LangChain memory)
- Hindi + Marathi language toggle
- PII masking before API call
- Session-only storage (no persistent document text in DB)
- Disclaimer UI on upload

Novel / differentiating (beyond existing work):

- Clause-level risk visualization (color-coded, not just a global label)
- India-specific legal domain: Rent Control Act, IPC references, RTI framework
- Document-grounded Q&A (answers must cite source clause, not give generic advice)
- Multilingual output for Hindi + Marathi (not English-only like all comparable tools)
- PII auto-redaction before external API call

---

9. EXISTING SYSTEMS SURVEYED (for literature table reference)

System	Type	Key Limitation
DoNotPay (USA)	Application	US law only; paid subscription
SpotDraft	Application	Enterprise-only; not for individuals
Manupatra / SCC Online	Application	For legal professionals; no plain-language output
ChatGPT / Gemini / Claude	Application	No dedicated upload UI; no India-specific focus; no persistent document Q&A

System	Type	Key Limitation
LawRato / Vakil Search	Application	Human lawyer dependency; no AI analysis
IBM Watson NLU	API/Tool	Enterprise pricing; no legal specialization
LegalEase (generic tools)	Application	English-only; global focus; no clause risk scoring

Note: 10 applications are covered. Research papers (30 minimum) need to be sourced from Google Scholar, IEEE Xplore, arXiv. See Section 12 for suggested search queries.

---

10. RESULT EVALUATION PARAMETERS

These are what the project will be measured against — important for both the survey table and project planning doc:

Parameter	Description	Target (Expected)
Simplification Accuracy	Does the plain-language output correctly represent the original clause? (human eval)	⩾ 85%
Risk Classification Accuracy	Does the risk label (L/M/H) match expert annotation?	⩾ 80%
BLEU Score	Machine translation quality for Hindi/Marathi output	⩾ 0.65
Response Latency	Time from upload to first result displayed	⩽ 5 seconds
User Comprehension Score	% of test users who correctly understood a clause after reading simplified version	⩾ 80%
PII Redaction Precision	% of real PII correctly masked before API call	⩾ 95%
System Uptime (Render)	Deployment reliability during testing period	⩾ 99%

---

11. DOCUMENTS ALREADY CREATED (in this conversation)

File	Contents
LexEase_Project_Documentation.docx	Full initial documentation (all 9 sections per professor's earlier format)
LexEase_Committee_QA_Prep.md	20 likely committee questions with full answers
LexEase_TechStack_and_Confidentiality.md	Tech stack justification vs alternatives + confidentiality architecture

---

12. RESEARCH PAPER SEARCH QUERIES (for literature survey)

Use these on **Google Scholar**, **IEEE Xplore**, **arXiv**, **Semantic Scholar**, and **ACL Anthology**:

1. "legal document simplification NLP"
2. "automated contract clause extraction deep learning"
3. "legal text summarization BERT"
4. "risk clause detection contracts machine learning"
5. "plain language transformation legal text"
6. "LLM legal document analysis"
7. "NLP Indian legal documents"
8. "information extraction legal contracts"
9. "explainable AI legal risk assessment"
10. "question answering legal documents transformer"
11. "named entity recognition legal text"
12. "text classification legal clauses"
13. "readability scoring legal documents"
14. "multilingual legal NLP"
15. "BERT fine-tuning legal domain"

Target journals/venues: ACL, EMNLP, NAACL, COLING, JURIX, Artificial Intelligence & Law (journal), IEEE Transactions on Neural Networks.

---

13. PROJECT MODULES (for planning document)

1. Document Ingestion Module — Upload, format validation, text extraction (PyMuPDF/pdfplumber)
2. Preprocessing Module — Text cleaning, PII masking, clause segmentation
3. AI Analysis Module — LangChain + Gemini: summarization, risk classification, Q&A
4. Multilingual Output Module — Google Translate API integration
5. Session Management Module — FastAPI session handling; PostgreSQL metadata storage
6. Frontend Presentation Module — React: upload UI, clause viewer, risk badges, chat interface
7. Evaluation Module — Test harness for accuracy, BLEU, latency benchmarking

---

14. DATASET

No single off-the-shelf dataset exists for Indian legal documents. Strategy:

- **Primary:** Collect 50–100 sample Indian legal PDFs (rent agreements, court notices, employment contracts) — these are publicly available / can be synthetically generated.
- **Secondary:** LEDGAR dataset (contract provisions, English), CUAD (Contract Understanding Atticus Dataset) — both open-access, for pretraining prompt calibration.
- **Annotation:** Team manually annotates 100 clauses with risk labels (Low/Medium/High) as ground truth for accuracy evaluation.

---

15. GIT / PROJECT STRUCTURE (recommended)

```
lexease/
├── frontend/              # React app
│   ├── src/
│   │   ├── components/    # Upload, ClauseCard, RiskBadge, ChatWindow
│   │   ├── pages/
│   │   └── api/           # Axios wrappers for backend calls
│   └── package.json
├── backend/               # FastAPI app
│   ├── app/
│   │   ├── routers/       # /upload, /analyze, /qa, /translate
```

```
│   │   ├── services/      # pdf_parser.py, ai_engine.py, pii_masker.py
│   │   ├── models/        # Pydantic schemas + DB models
│   │   └── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── docs/                  # All professor-required documentation
│   ├── literature_survey.docx
│   ├── project_plan.docx
│   └── context_snapshot.md  ← (this file)
├── dataset/               # Sample documents and annotations (gitignored if
sensitive)
├── tests/                 # pytest unit + integration tests
├── docker-compose.yml     # Spins up backend + PostgreSQL locally
├── render.yaml            # Render deployment config
├── .env.example           # Template (never commit real .env)
├── .gitignore
└── README.md
```

Git workflow for 5 members:

- Branch naming: `feature/<member-initials>/<short-description>` e.g. `feature/ak/pdf-parser`
- One branch per feature/module. PRs must be reviewed by at least 1 other member.
- `main` = always deployable. `develop` = integration branch. Features merge into `develop` → `main`.
- Commit messages: `[MODULE] short description` e.g. `[BACKEND] add clause segmentation endpoint`
- GitHub Issues: one issue per task; label by module (frontend, backend, ai, docs, testing)

---

16. DEPLOYMENT PLAN

Development:

- Docker Compose locally: `docker compose up` starts FastAPI + PostgreSQL together
- Each member runs the full stack locally via Docker — no "works on my machine" problems

Staging/Production (Render free tier):

- Backend: Render Web Service (Python/FastAPI) — connect GitHub repo, set env vars

in Render dashboard

- Frontend: Render Static Site (React build) — or Vercel (also free, better for React)
- Database: Render PostgreSQL (free tier: 90-day database, sufficient for academic use)
- Environment variables: GEMINI_API_KEY, DATABASE_URL, ALLOWED_ORIGINS — set in Render dashboard, never in code
- `render.yaml` in repo root: defines all services so deployment is one-click

Pre-demo checklist:

- Ping the Render URL ~5 minutes before demo (free tier sleeps after 15 min inactivity — first wake takes 30–60 sec)
- Use sample/synthetic legal documents for the live demo — never real personal documents

---

17. PUBLICATION PLAN (if applicable)

If the team wants to publish the project:

- **Target venue:** IJRASET, IRJET, IJCRT (Indian student-friendly journals, indexed) — lower bar, good for final year publication proof
- **Better target:** IEEE INDICON, ICDCN, or ACL Student Research Workshop — competitive but prestigious
- **Paper angle:** "Clause-Level Risk Analysis of Indian Legal Documents Using LLM-Based Prompt Engineering" — focuses on the India-specific + clause-level novelty
- **Preprint:** arXiv cs.CL — free, instant visibility, counts for academic portfolios
- Start the paper draft in Semester 7 alongside development; results section filled in after evaluation in Semester 8

---

18. KEY CONSTRAINTS TO NEVER VIOLATE

1. No paid APIs or software — everything must work on free tiers.
2. No GPU or expensive hardware — all AI is API-based (Gemini).
3. Render, not Railway (Railway has no real free tier as of 2023).
4. Do NOT store document text in the database — session memory only.
5. Always carry the disclaimer: "Academic prototype — do not upload real sensitive documents."
6. Use sample documents for demos, not real personal legal documents.

7. The free Gemini tier CAN use your data for training — be honest about this, don't oversell privacy.

END OF CONTEXT SNAPSHOT

Reference this file at the start of every Claude Code session.